



ESCUELA SUPERIOR DE ECONOMÍA Y NEGOCIOS INGENIERÍA DE SOFTWARE Y NEGOCIOS DIGITALES

CICLO 03-2026

GUIA DE LABORATORIO N° 3

Nombre de la practica: Posicionamiento, visualización y maquetación de sitios web

Tiempo estimado: 4 horas

Materia: Desarrollo Web I

I. Objetivos

Que el estudiante sea capaz de:

1. Utilizar los diferentes tipos de posicionamiento CSS y diferenciarlos apropiadamente.
2. Utilizar las propiedades CSS relacionada con la visualización de elementos.
3. Maquetar sitios web con diseños flexibles mediante el uso de las propiedades de Flexbox

II. Introducción Teórica

Para crear una buena composición de los elementos en pantalla, es necesario conocer todas las opciones posibles al posicionamiento de elementos y, sobre todo, tener algunas nociones de los tipos de elementos con los que trabaja CSS a la hora de posicionarlos en pantalla.

Para cumplir con el box model los navegadores crean una caja para representar cada elemento de una página HTML. Para hacerlo, los navegadores llevan a cabo una compleja operación en la que se tienen en cuenta los siguientes parámetros:

1. Las propiedades width (ancho) y height (alto) de las cajas, en caso de haber especificado unas dimensiones específicas.
2. El tipo de cada elemento HTML, bien si es un elemento de bloque o un elemento de línea.
3. Posicionamiento de las cajas (estático, relativo, absoluto, fijo o flotante).
4. El parentesco y descendencia de cada elemento.
5. Las dimensiones de la ventana del navegador en función de la resolución de pantalla o las dimensiones de las imágenes, entre otros.

A este procedimiento se le conoce comúnmente como “*modelo de formato visual*”, que definido para entenderlo rápidamente puede ser algo así como el modelo que siguen los navegadores para procesar el contenido de un documento para los medios visuales.

Existen dos grandes clasificaciones de los elementos HTML que distingue un navegador:

- **Elementos de bloque**

Los elementos de bloque ("block elements" en inglés) siempre empiezan en una nueva línea, ocupan todo el espacio disponible hasta el final de la línea y pueden contener otros elementos de bloque o de línea.

- **Elementos de línea**

Los elementos de línea ("inline elements" en inglés) no empiezan necesariamente en nueva línea, sólo ocupan el espacio necesario para mostrar sus contenidos, y solo pueden contener otros elementos de línea o contenido.

Algunos ejemplos de elementos de bloque son: *header, nav, aside, section, article, div, p, footer, h1, h2, h3, h4, h5, h6*, etc.

Por su parte, algunos ejemplos de elementos de línea son: *a, b, i, br, cite, img, span, sup, sub*, etc.

Posicionamiento

Los navegadores crean y posicionan de forma automática todas las cajas que forman cada página HTML. No obstante, CSS permite al diseñador modificar la posición en la que se muestra cada caja.

Utilizando las propiedades que proporciona CSS para alterar la posición de las cajas es posible realizar efectos muy avanzados y diseñar estructuras de páginas que de otra forma no serían posibles.

El estándar de CSS define cinco modelos diferentes para posicionar una caja:

- **Posicionamiento normal o estático.** Se trata del posicionamiento que utilizan los navegadores si no se indica lo contrario.
- **Posicionamiento relativo:** variante del posicionamiento normal que consiste en posicionar una caja según el posicionamiento normal y después desplazarla respecto de su posición original.
- **Posicionamiento absoluto:** la posición de una caja se establece de forma absoluta respecto de su elemento contenedor y el resto de elementos de la página ignoran la nueva posición del elemento.
- **Posicionamiento fijo:** variante del posicionamiento absoluto que convierte una caja en un elemento inamovible, de forma que su posición en la pantalla siempre es la misma independientemente del resto de elementos e independientemente de si el usuario sube o baja la página en la ventana del navegador.
- **Posicionamiento flotante:** se trata del modelo más especial de posicionamiento, ya que desplaza las cajas todo lo posible hacia la izquierda o hacia la derecha de la línea en la que se encuentran.

Propiedad position

El posicionamiento de una caja se establece mediante la propiedad *position*:

position	Posicionamiento
Valores	<code>static</code> <code>relative</code> <code>absolute</code> <code>fixed</code> <code>inherit</code>
Se aplica a	Todos los elementos
Valor inicial	<code>static</code>
Descripción	Selecciona el posicionamiento con el que se mostrará el elemento

El significado de cada uno de los posibles valores de la propiedad *position* es el siguiente:

- **static:** corresponde al posicionamiento normal o estático. Si se utiliza este valor, se ignoran los valores de las propiedades *top*, *right*, *bottom* y *left* que se verán a continuación.
- **relative:** corresponde al posicionamiento relativo. El desplazamiento de la caja se controla con las propiedades *top*, *right*, *bottom* y *left*.
- **absolute:** corresponde al posicionamiento absoluto. El desplazamiento de la caja también se controla con las propiedades *top*, *right*, *bottom* y *left*, pero su interpretación es mucho más compleja, ya que el origen de coordenadas del desplazamiento depende del posicionamiento de su elemento contenedor.
- **fixed:** corresponde al posicionamiento fijo. El desplazamiento se establece de la misma forma que en el posicionamiento absoluto, pero en este caso el elemento permanece inamovible en la pantalla.

La propiedad *position* no permite controlar el posicionamiento flotante, que se establece con otra propiedad llamada *float* y que se explica más adelante. Además, la propiedad *position* sólo indica cómo se posiciona una caja, pero no la desplaza.

Propiedades de desplazamiento

Normalmente, cuando se posiciona una caja también es necesario desplazarla respecto de su posición original o respecto de otro origen de coordenadas. CSS define cuatro propiedades llamadas *top*, *right*, *bottom* y *left* para controlar el desplazamiento de las cajas posicionadas:

top right bottom left	Desplazamiento superior Desplazamiento lateral derecho Desplazamiento inferior Desplazamiento lateral izquierdo
Valores	<code><medida></code> <code><porcentaje></code> <code>auto</code> <code>inherit</code>
Se aplica a	Todos los elementos posicionados
Valor inicial	<code>auto</code>
Descripción	Indican el desplazamiento horizontal y vertical del elemento respecto de su posición original

En el caso del posicionamiento relativo, cada una de estas propiedades indica el desplazamiento del elemento desde la posición original de su borde superior/derecho/inferior/izquierdo. Si el posicionamiento es absoluto, las propiedades indican el desplazamiento del elemento respecto del borde superior/derecho/inferior/izquierdo de su primer elemento padre posicionado.

En cualquiera de los dos casos, si el desplazamiento se indica en forma de porcentaje, se refiere al porcentaje sobre la anchura (propiedades right y left) o altura (propiedades top y bottom) del elemento.

Visualización

Además de las propiedades que controlan el posicionamiento de los elementos, CSS define otras cuatro propiedades para controlar su visualización: `display`, `visibility`, `overflow` y `z-index`.

Utilizando algunas de estas propiedades es posible ocultar y/o hacer invisibles las cajas de los elementos, por lo que son imprescindibles para realizar efectos avanzados.

- **Propiedad display**

Es la primera de las propiedades de visualización en CSS y es una de las más útiles para diseñadores.

display	Visualización de un elemento
Valores	<code>inline</code> <code>block</code> <code>none</code> <code>list-item</code> <code>run-in</code> <code>inline-block</code> <code>table</code> <code>inline-table</code> <code>table-row-group</code> <code>table-header-group</code> <code>table-footer-group</code> <code>table-row</code> <code>table-column-group</code> <code>table-column</code> <code>table-cell</code> <code>table-caption</code> <code>inherit</code>
Se aplica a	Todos los elementos
Valor inicial	<code>inline</code>
Descripción	Permite controlar la forma de visualizar un elemento e incluso ocultarlo

Los valores más utilizados son `inline`, `block`, `none` e `inline-block`.

- ✓ El valor `block` permite mostrar un elemento como si fuera un elemento de bloque, independientemente del tipo de elemento que se trate.
- ✓ El valor `inline` permite visualizar un elemento en forma de elemento en línea, independientemente del tipo de elemento que se trate.
- ✓ El valor `none` permite ocultar un elemento y hacer que desaparezca de la página. El resto de elementos se visualizan como si no existiera el elemento oculto, es decir, pueden ocupar el espacio en el que se debería visualizar el elemento.
- ✓ El valor `inline-block` es muy interesante ya que permite obtener un comportamiento intermedio entre los elementos de línea y los de bloque. **Los elementos inline-block fluyen con el texto y demás elementos** como si fueran elementos en-línea **y además respetan el ancho, el alto y los márgenes verticales**. Son lo mejor de los dos mundos. Con este valor se pueden conseguir comportamientos similares que con el uso de la propiedad `float` pero sin el dolor de cabeza de tener que usar propiedades como `clear` u `overflow`.

- **Propiedad visibility**

Mediante el uso de las propiedades `visibility` es posible ocultar un elemento manteniendo intacto su espacio (al contrario de lo que ocurriría con `display: none`). Con esta propiedad es posible ocultar elementos del diseño sin afectar al resto de elementos.

visibility	Visibilidad de un elemento
Valores	<code>visible</code> <code>hidden</code> <code>collapse</code> <code>inherit</code>
Se aplica a	Todos los elementos
Valor inicial	<code>visible</code>
Descripción	Permite hacer visibles e invisibles a los elementos

- **Propiedad overflow**

Mediante el uso de esta propiedad es posible controlar el comportamiento de los contenidos cuando éstos se salen de su elemento contenedor. En el diseño mediante el modelo de caja es habitual que el contenido se desborde fuera de su elemento contenedor. Esto ocurre cuando al elemento contenedor se le aplican unas dimensiones concretas mediante el uso de las propiedades `width` y `height`.

overflow	Parte sobrante de un elemento
Valores	<code>visible</code> <code>hidden</code> <code>scroll</code> <code>auto</code> <code>inherit</code>
Se aplica a	Elementos de bloque y celdas de tablas
Valor inicial	<code>visible</code>
Descripción	Permite controlar los contenidos sobrantes de un elemento

Los valores de la propiedad `overflow` tienen el siguiente significado:

- ✓ **visible:** el contenido no se corta y se muestra sobresaliendo la zona reservada para visualizar el elemento. Este es el comportamiento por defecto.
- ✓ **hidden:** el contenido sobrante se oculta y sólo se visualiza la parte del contenido que cabe dentro de la zona reservada para el elemento.
- ✓ **scroll:** solamente se visualiza el contenido que cabe dentro de la zona reservada para el elemento, pero también se muestran barras de *scroll* que permiten visualizar el resto del contenido.

- **Propiedad z-index**

Además de posicionar una caja de forma horizontal y vertical, CSS permite controlar la posición tridimensional de las cajas posicionadas. De esta forma, es posible controlar las cajas que se muestran delante o detrás de otras cajas cuando se producen solapamientos.

La posición tridimensional se establece sobre un tercer eje llamado Z y se controla mediante la propiedad `z-index`. Utilizando esta propiedad es posible crear páginas complejas con varios niveles o capas.

A continuación se muestra la definición formal de la propiedad z-index:

z-index	Orden tridimensional
Valores	auto <numero> inherit
Se aplica a	Elementos que han sido posicionados explícitamente
Valor inicial	auto
Descripción	Establece el nivel tridimensional en el que se muestra el elemento

Cuanto más alto sea el valor numérico, más cerca del usuario se muestra la caja. Un elemento con z-index: 10 se muestra por encima de los elementos con z-index: 8 o z-index: 9, pero por debajo de elementos con z-index: 20 o z-index: 50.

Modelo de maquetación flexible (Flexbox)

Flexbox, abreviatura de Flexible Box Layout, es un modelo de CSS diseñado para distribuir y alinear elementos dentro de un contenedor. Resulta especialmente útil cuando los elementos deben organizarse en una fila o en una columna y el espacio disponible puede variar.

Contenedor flexible y elementos flexibles

Flexbox comienza al asignar display: flex a un elemento. Este elemento se convierte en el contenedor flexible y sus hijos directos pasan a ser elementos flexibles. Los descendientes que no sean hijos directos no se distribuyen automáticamente mediante Flexbox.

```
.contenedor {  
  display: flex;  
}
```

En el siguiente fragmento, los tres elementos article son hijos directos de section. Por tanto, serán distribuidos como elementos flexibles.

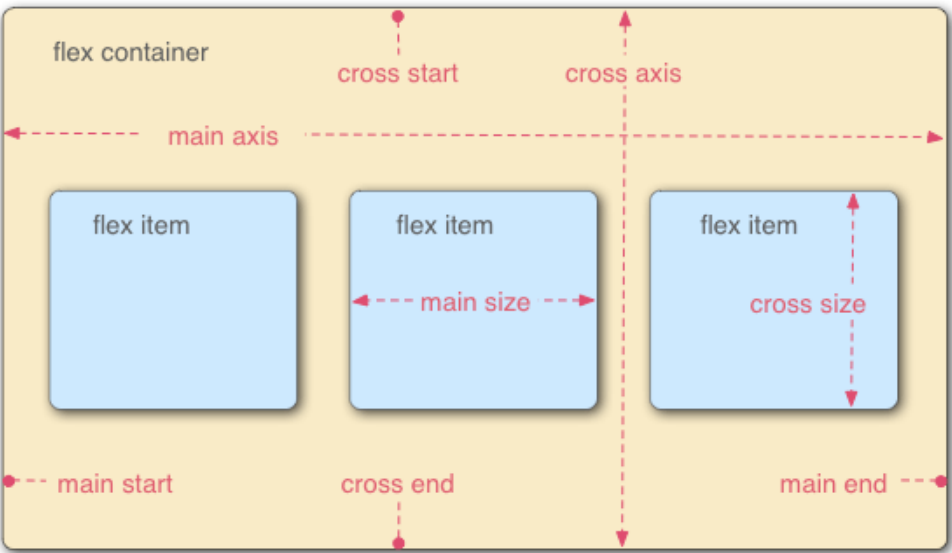
```
<section class="contenedor">  
  <article>Primer elemento</article>  
  <article>Segundo elemento</article>  
  <article>Tercer elemento</article>  
</section>
```

Eje principal y eje transversal

La distribución de Flexbox se interpreta a partir de dos ejes. El eje principal determina la dirección en la que se colocan los elementos. El eje transversal es perpendicular al eje principal.

- 1. Eje principal.** Sigue la dirección establecida mediante flex-direction. Cuando se utiliza row, se extiende horizontalmente; cuando se utiliza column, se extiende verticalmente.
- 2. Eje transversal.** Se utiliza para alinear los elementos en la dirección perpendicular al eje principal.

Esta distinción permite interpretar correctamente las propiedades de alineación. justify-content actúa sobre el eje principal, mientras que align-items actúa sobre el eje transversal.



Dirección de los elementos

La propiedad flex-direction establece la dirección del eje principal y, por tanto, la dirección en la que se colocan los elementos.

Valor	Comportamiento
row	Coloca los elementos en una fila. Es el valor predeterminado.
row-reverse	Coloca los elementos en una fila e invierte su orden visual.
column	Coloca los elementos en una columna.
column-reverse	Coloca los elementos en una columna e invierte su orden visual.

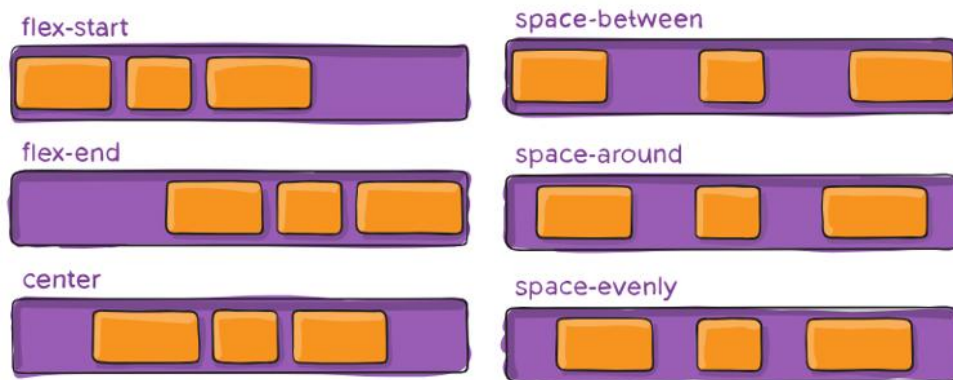
```
.contenedor {
  display: flex;
  flex-direction: row;
}
```

Distribución sobre el eje principal

La propiedad `justify-content` distribuye los elementos y el espacio libre disponible sobre el eje principal. Su efecto depende de la dirección establecida mediante `flex-direction`.

1. **flex-start.** Agrupa los elementos al inicio del eje principal.
2. **center.** Agrupa los elementos en el centro del eje principal.
3. **flex-end.** Agrupa los elementos al final del eje principal.
4. **space-between.** Distribuye el espacio libre entre los elementos, sin dejar espacio adicional en los extremos.
5. **space-around.** Distribuye espacio alrededor de cada elemento. El espacio de los extremos es menor que el espacio interior.
6. **space-evenly.** Distribuye espacios iguales entre los elementos y en ambos extremos.

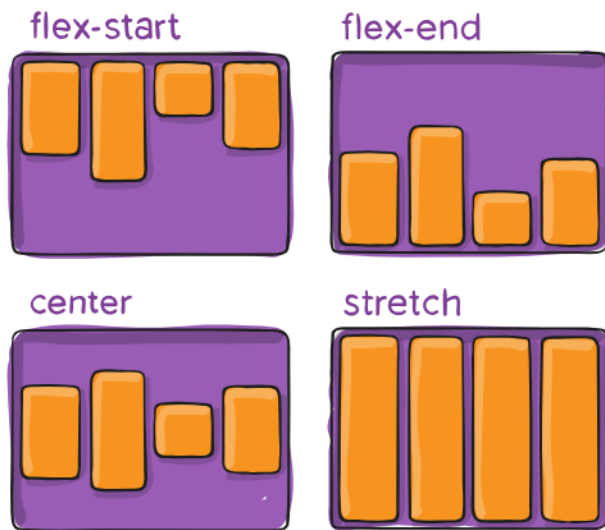
```
.contenedor {  
  display: flex;  
  justify-content: space-between;  
}
```



Alineación sobre el eje transversal

La propiedad `align-items` controla la alineación de todos los elementos sobre el eje transversal. Entre sus valores más frecuentes se encuentran `stretch`, `flex-start`, `center` y `flex-end`. El valor `stretch`, utilizado de forma predeterminada, puede hacer que los elementos ocupen todo el espacio disponible en el eje transversal cuando no poseen una dimensión definida.

```
.contenedor {  
  display: flex;  
  align-items: center;  
}
```



Cuando un solo elemento necesita una alineación diferente, puede aplicarse `align-self` directamente sobre ese elemento.

```
.elemento-destacado {  
  align-self: flex-end;  
}
```

Distribución en varias líneas

De forma predeterminada, Flexbox intenta mantener todos los elementos en una sola línea. La propiedad `flex-wrap` permite que los elementos pasen a una línea adicional cuando el espacio disponible resulta insuficiente.

```
.contenedor {  
  display: flex;  
  flex-wrap: wrap;  
}
```

El valor `wrap` es especialmente útil para conjuntos de tarjetas. Permite que la cantidad de columnas cambie según el ancho disponible sin utilizar `float` ni efectuar cálculos manuales para cada fila.

Separación entre elementos

La propiedad `gap` define una separación uniforme entre los elementos flexibles. A diferencia de los márgenes, no agrega espacio en los bordes exteriores del contenedor ni obliga a corregir el ancho del primer o del último elemento.

```
.contenedor {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}
```

Tamaño flexible de los elementos

La propiedad abreviada `flex` permite controlar cuánto puede crecer un elemento, cuánto puede reducirse y cuál es su tamaño inicial. Su sintaxis general es `flex: crecimiento reducción tamaño-inicial`.

```
.tarjeta {  
  flex: 1 1 280px;  
}
```

En este caso, cada tarjeta puede crecer para aprovechar el espacio disponible, puede reducirse cuando sea necesario y parte de un tamaño base de 280 píxeles. En combinación con `flex-wrap`, esta declaración permite formar varias columnas sin asignar un porcentaje rígido a cada tarjeta.

1. **flex-grow.** Indica la proporción de espacio libre que puede recibir el elemento.
2. **flex-shrink.** Indica la capacidad del elemento para reducir su tamaño.
3. **flex-basis.** Establece el tamaño inicial utilizado antes de distribuir el espacio disponible.

Consideraciones de uso

1. **Flexbox distribuye principalmente en una dimensión.** Resulta apropiado cuando el interés principal está en una fila o una columna. Para distribuciones bidimensionales complejas puede resultar más adecuado CSS Grid.
2. **El orden visual no debe reemplazar el orden lógico.** Propiedades como `order` y los valores `reverse` pueden modificar la presentación sin cambiar el orden del documento. Deben utilizarse con cautela para no afectar la lectura mediante teclado o tecnologías de asistencia.
3. **Las dimensiones fijas deben utilizarse con moderación.** Una altura fija puede cortar contenido cuando cambia el texto o el tamaño de la pantalla. Siempre que sea posible, debe permitirse que la altura dependa del contenido.
4. **Flexbox no convierte por sí solo una página en completamente adaptable.** Propiedades como `flex-wrap` y `flex-basis` ofrecen flexibilidad básica, pero algunos cambios de diseño requieren consultas de medios, las cuales se estudiarán posteriormente.

EJEMPLO 1. MARAVILLAS DEL MUNDO

En este ejemplo se construirá una página dedicada a las siete maravillas del mundo moderno. Durante el procedimiento se modificará temporalmente el diseño para estudiar el flujo normal, Flexbox, el posicionamiento relativo, absoluto, adherente y fijo, el orden de superposición y las propiedades utilizadas para ocultar elementos.

Preparación de los archivos

1. Crear una carpeta con el nombre: **NºCarnet_Guia3**, Nota: Cambiar NºCarnet por su número de carnet de la Universidad. Dentro de esa carpeta, extraiga los recursos proporcionados por su docente.

Estructura HTML

2. Dentro de la carpeta **ejemplo1** debe crear el archivo **index.html** con el siguiente contenido:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Recorrido visual por las siete maravillas
del mundo moderno.">
  <title>Maravillas del Mundo</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body id="inicio">
  <header class="encabezado-principal">
    <div class="contenedor barra-navegacion">
      <a class="marca" href="#inicio">Maravillas del Mundo</a>

      <nav aria-label="Navegación principal">
        <ul class="menu">
          <li><a href="#maravillas">Maravillas</a></li>
          <li><a href="#historia">Historia</a></li>
          <li><a href="#creditos">Créditos</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <main>
    <section class="presentacion">
      <div class="contenedor presentacion-contenido">
```

```

<p class="sobretitulo">Patrimonio y cultura</p>
<h1>Siete lugares que transformaron la historia</h1>
<p>
    Un recorrido por construcciones extraordinarias que reflejan el
ingenio,
    las creencias y la identidad de distintas civilizaciones.
</p>
<a class="boton-principal" href="#maravillas">Iniciar recorrido</a>
</div>
</section>

<section id="maravillas" class="seccion-maravillas contenedor">
<header class="titulo-seccion">
<p class="sobretitulo">Las siete elegidas</p>
<h2>Maravillas del mundo moderno</h2>
<p>Cada monumento conserva una historia vinculada con su época y su
cultura.</p>
</header>

<div class="maravillas">
<article class="maravilla">
<div class="maravilla-imagen">
<span class="continente">Asia</span>

</div>
<div class="maravilla-contenido">
<p class="ubicacion">China</p>
<h3>Gran Muralla China</h3>
<p>Sistema de fortificaciones construido y ampliado durante
distintos períodos de la historia china.</p>
<a href="https://whc.unesco.org/en/list/438/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
</div>
</article>

<article class="maravilla">
<div class="maravilla-imagen">
<span class="continente">Asia</span>

</div>
<div class="maravilla-contenido">
<p class="ubicacion">Jordania</p>
<h3>Petra</h3>

```

```

        <p>Ciudad nabatea tallada en piedra y conectada mediante rutas
comerciales del mundo antiguo.</p>
        <a href="https://whc.unesco.org/en/list/326/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
    </div>
</article>

<article class="maravilla">
    <div class="maravilla-imagen">
        <span class="continente">Europa</span>
        
    </div>
    <div class="maravilla-contenido">
        <p class="ubicacion">Italia</p>
        <h3>Coliseo de Roma</h3>
        <p>Anfiteatro del siglo I que se convirtió en uno de los símbolos
más reconocibles del Imperio romano.</p>
        <a href="https://whc.unesco.org/en/list/91/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
    </div>
</article>

<article class="maravilla">
    <div class="maravilla-imagen">
        <span class="continente">América</span>
        
    </div>
    <div class="maravilla-contenido">
        <p class="ubicacion">México</p>
        <h3>Chichén Itzá</h3>
        <p>Centro ceremonial maya cuya arquitectura integra conocimientos
astronómicos y una compleja vida política.</p>
        <a href="https://whc.unesco.org/en/list/483/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
    </div>
</article>

<article class="maravilla">
    <div class="maravilla-imagen">
        <span class="continente">América</span>
        
    </div>

```

```

        <div class="maravilla-contenido">
            <p class="ubicacion">Perú</p>
            <h3>Machu Picchu</h3>
            <p>Ciudad inca construida en los Andes mediante una notable
adaptación de la arquitectura al paisaje.</p>
            <a href="https://whc.unesco.org/en/list/274/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
        </div>
    </article>

    <article class="maravilla">
        <div class="maravilla-imagen">
            <span class="continente">Asia</span>
            
        </div>
        <div class="maravilla-contenido">
            <p class="ubicacion">India</p>
            <h3>Taj Mahal</h3>
            <p>Mausoleo de mármol blanco que reúne tradiciones arquitectónicas
persas, islámicas e indias.</p>
            <a href="https://whc.unesco.org/en/list/252/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
        </div>
    </article>

    <article class="maravilla">
        <div class="maravilla-imagen">
            <span class="continente">América</span>
            
        </div>
        <div class="maravilla-contenido">
            <p class="ubicacion">Brasil</p>
            <h3>Cristo Redentor</h3>
            <p>Monumento art déco levantado sobre el Corcovado y convertido en
símbolo cultural de Río de Janeiro.</p>
            <a href="https://whc.unesco.org/en/list/1100/" target="_blank"
rel="noopener noreferrer">Conocer su historia</a>
        </div>
    </article>
</div>
</section>

<section id="historia" class="seccion-historia">

```

```

<div class="contenedor historia-contenido">
  <div>
    <p class="sobretitulo">Una selección contemporánea</p>
    <h2>Patrimonio elegido por votación mundial</h2>
  </div>
  <p>
    La lista fue anunciada en 2007 después de una votación internacional.
    Aunque reúne lugares de épocas y culturas diferentes, todos
representan
    logros arquitectónicos que continúan despertando interés en la
actualidad.
  </p>
</div>
</section>
</main>

<a class="volver-arriba" href="#inicio" aria-label="Volver al inicio">↑</a>

<footer id="creditos" class="pie-pagina">
  <div class="contenedor pie-contenido">
    <p><strong>Maravillas del Mundo</strong></p>
    <p>Imágenes de Wikimedia Commons.</p>
  </div>
</footer>
</body>
</html>

```

Estilos iniciales

3. Dentro de la carpeta **ejemplo1/css**, crear un archivo con el nombre **style.css**. Comencemos definiendo nuestra paleta de colores mediante variables

```

:root {
  --color-texto: #342e29;
  --color-fondo: #f5efe6;
  --color-principal: #293e36;
  --color-acento: #93472f;
  --color-superficie: #ffffff;
  --color-contraste: #ead1ad;
  --color-borde: #ded3c4;
  --color-fondo-secundario: #eadfce;
  --color-capa-inicial: rgba(25, 35, 29, 0.85);
  --color-capa-final: rgba(25, 35, 29, 0.65);
  --color-sombra: rgba(52, 46, 41, 0.10);
}

```

4. Escriba una regla para body que:
 - a) Elimine el margen predeterminado.
 - b) Utilice "Segoe UI", Arial, sans-serif.
 - c) Establezca un interlineado de 1.6.
 - d) Use las variables correspondientes para el color de texto y el fondo.

Actualice la página y compruebe los cambios.

5. A continuación, vamos a centrar el contenido:

```
.contenedor {  
  width: 90%;  
  max-width: 1140px;  
  margin: 0 auto;  
}
```

Reduzca y amplíe la ventana. Observe que el contenedor utiliza el 90 % del espacio disponible hasta alcanzar un máximo de 1140 píxeles.

6. Ahora vamos a limitar el ancho de las imágenes evitando que estas sean más anchas que su contenedor.

```
img {  
  max-width: 100%;  
}
```

7. Configure los estilos de texto. Consiga los siguientes resultados:
 - a) h1, h2 y h3 deben utilizar un interlineado de 1.2.
 - b) h2 y h3 deben usar --color-principal.
 - c) Los enlaces deben usar --color-acentos.
 - d) Durante :hover y :focus-visible, los enlaces deben cambiar a --color-principal.

Encabezado y menú

8. Aplique los colores del encabezado:

```
.encabezado-principal {  
  color: var(--color-superficie);  
  background-color: var(--color-principal);  
}  
  
.marca {  
  color: var(--color-superficie);  
  font-size: 1.15rem;  
  font-weight: bold;  
  text-decoration: none;  
}
```

Hasta este punto, la marca y la navegación todavía siguen el flujo normal y aparecen una debajo de la otra.

9. Distribuya la barra con Flexbox. Convierta `.barra-navegacion` en un contenedor flexible. Después, agregue las propiedades necesarias para:
 - a) Colocar la marca y la navegación en extremos opuestos.
 - b) Alinearlas en el centro del eje transversal.
 - c) Permitir el salto de línea (`flex-wrap`).
 - d) Aplicar un gap de `0.75rem` a `2rem`.
 - e) Aplicar `1rem` de relleno vertical.

Visualice el resultado antes de continuar.

10. Organice el menú: elimine el margen, el relleno y las viñetas de `.menu`. Luego, convierta la lista en un contenedor flexible con una separación de `1rem` y permita el salto de línea. Configure sus enlaces con texto blanco y sin subrayado. Durante `:hover` y `:focus-visible`, utilice `--color-contraste`.

Reduzca el ancho de la ventana y observe el comportamiento del menú.

Presentación principal

11. Coloque la fotografía de fondo mediante las siguientes reglas de estilo:

```
.presentacion {  
  padding: 4rem 0;  
  color: var(--color-superficie);  
  background-color: var(--color-principal);  
  background-image: url("../images/gran-muralla-china.jpg");  
  background-size: cover;  
  background-position: center;  
}
```

12. Agregaremos una capa de contraste que mejore la legibilidad del texto sobre las zonas claras y oscuras de la fotografía. Sustituya únicamente la declaración `background-image` por:

```
background-image:  
  linear-gradient(  
    to right,  
    var(--color-capa-inicial),  
    var(--color-capa-final)  
  ),  
  url("../images/gran-muralla-china.jpg");
```

La primera capa se dibuja sobre la segunda. El degradado oscurece la fotografía, pero no cambia la opacidad del texto.

13. Formatee el contenido de presentación. Agregue las reglas necesarias para obtener:
 - a) Un h1 con ancho máximo de 760px, margen inferior de 1rem y fuente de 2.5rem.
 - b) Párrafos con ancho máximo de 640px.
 - c) Un .sobretitulo de 0.85rem, en negrita, mayúsculas y color de acento.
 - d) El sobretítulo de la presentación en --color-contraste.
14. Construya el botón principal. Estilice .boton-principal como un elemento inline-block con:
 - a) Margen superior de 1rem.
 - b) Relleno de 0.75rem 1.25rem.
 - c) Texto blanco sobre fondo de acento.
 - d) Radio de borde de 0.3rem.
 - e) Ausencia de subrayado.

Durante :hover y :focus-visible, intercambie los colores de texto y fondo por --color-principal y --color-contraste respectivamente.

Construcción de la galería

15. Prepare la sección. Asigne 3rem de relleno vertical a .seccion-maravillas. Limite .titulo-seccion a 680px y aplique 2rem de margen inferior. Configure su h2 con margen 0 0 0.75rem y fuente de 2rem.
16. Active Flexbox sin completar la distribución. Agregue solamente:

```
.maravillas {  
  display: flex;  
}
```

Visualice el resultado.

- a) ¿Cuántas filas intenta formar Flexbox?
- b) ¿Qué ocurre con el ancho de las tarjetas?
- c) ¿El contenido permanece cómodo de leer?

No corrija todavía el problema.

17. Permita el salto de línea. Agregue a .maravillas la propiedad de flexbox que permite distribuir los artículos en varias filas. Visualice nuevamente la página y explique qué cambió. Si las tarjetas todavía no poseen un tamaño adecuado, continúe con el paso siguiente.
18. Establezca un tamaño flexible. Agregue:

```
.maravilla {  
  flex: 1 1 300px;  
}
```

Los valores indican que la tarjeta puede crecer, puede reducirse y parte de un tamaño base de 300 píxeles. Cambie el ancho del navegador y observe cómo varía la cantidad de tarjetas por fila.

19. Separe las tarjetas. Agregue `gap: 1.5rem` a `.maravillas`. Explique la diferencia entre el espacio creado por `gap` y el relleno interior que posteriormente se aplicará a cada tarjeta.

20. Observe el crecimiento de la última tarjeta. Amplíe la ventana hasta que la última fila quede incompleta. Observe el ancho de la última tarjeta.

- a) ¿Por qué crece más que las tarjetas de una fila completa?
- b) ¿Qué valor de `flex` permite ese crecimiento?
- c) ¿Resulta visualmente conveniente que ocupe todo el espacio?

Limite las tarjetas con `max-width: 360px`. Después, determine qué propiedad de Flexbox permite centrar los elementos de una fila incompleta y agréguela a `.maravillas`.

Apariencia de las tarjetas

21. Agregue la superficie de la tarjeta. Incorpore a `.maravilla`:

```
background-color: var(--color-superficie);
border: 1px solid var(--color-borde);
border-radius: 0.5rem;
box-shadow: 0 4px 12px var(--color-sombra);
```

Observe las esquinas superiores. Aunque la tarjeta tiene bordes redondeados, las fotografías todavía pueden cubrir visualmente ese redondeo.

22. Controle el contenido que sobresale. Agregue a `.maravilla`:

```
overflow: hidden;
```

Visualice los cambios y explique por qué ahora se respetan las esquinas redondeadas.

23. Uniforme las dimensiones de las fotografías. Agregue:

```
.maravilla img {
  display: block;
  width: 100%;
  height: 220px;
}
```

Observe las siete fotografías. Preste especial atención a Petra.

- a) ¿Mantienen su proporción?
- b) ¿Qué diferencia se observa entre la fotografía vertical y las horizontales?

24. Compare los valores de object fit. Agregue temporalmente:

```
object-fit: contain;
```

Observe que la fotografía completa cabe dentro del área, pero pueden aparecer espacios libres.

Cambie el valor por:

```
object-fit: cover;
```

Compare ambos resultados y conserve cover. Explique qué parte del contenido puede perderse al utilizar este valor.

25. Formatee el contenido de las tarjetas. Aplique 1.25rem de relleno a .maravilla-contenido.

Consiga además:

- a) Una .ubicación sin margen, de 0.85rem de tamaño de fuente, en negrita y color de acento.
- b) Un h3 de 1.35rem con margen 0.25rem 0 0.75rem.
- c) Enlaces en negrita.

26. Compare la posición de los enlaces. Observe las tarjetas de una misma fila. Las descripciones poseen longitudes diferentes y los enlaces no necesariamente terminan a la misma altura.

Convierta .maravilla y .maravilla-contenido en contenedores flexibles verticales. Permita que el contenido crezca mediante flex-grow: 1.

Después, agregue al enlace:

```
margin-top: auto;  
align-items: flex-start;
```

Visualice el resultado. Explique qué espacio absorbe el margen automático.

Posicionamiento relativo

27. Desplace temporalmente la segunda tarjeta. Agregue al final de la hoja:

```
.maravilla:nth-child(2) {  
  position: relative;  
  left: 50px;  
  top: 30px;  
  z-index: 2;  
}
```

Visualice el resultado.

- a) ¿En qué dirección se desplazó la tarjeta?
- b) ¿Desde qué posición se calcula el desplazamiento?
- c) ¿Las demás tarjetas ocuparon su posición original?
- d) ¿La tarjeta desplazada puede superponerse con otras?

En el posicionamiento relativo, el elemento conserva el espacio que ocupaba en el flujo normal.

28. Compruebe el efecto de las coordenadas. Cambie temporalmente left: 50px por right: 50px.

- a) ¿La tarjeta se desplaza hacia la derecha o hacia la izquierda?
- b) ¿Por qué una coordenada situada en un lado puede producir movimiento en la dirección contraria?

Restituya left: 50px antes de continuar.

Posicionamiento absoluto

29. Cambie el tipo de posicionamiento. En la regla experimental del segundo artículo, sustituya:

position: relative;

por:

position: absolute;

Visualice los cambios.

- a) ¿Qué ocurrió con las demás tarjetas?
- b) ¿Continúa reservado el espacio original de Petra?
- c) ¿Respecto de qué área parecen calcularse top y left?

Un elemento con posicionamiento absoluto sale del flujo normal. Sus coordenadas se calculan respecto del antecesor posicionado más cercano. Si no existe, se utiliza el área inicial del documento.

30. **Cambie el sistema de referencia.** Consiga que .maravillas sea el antecesor posicionado de la tarjeta absoluta sin desplazar el contenedor de su ubicación normal.

Visualice la página.

- a) ¿Desde dónde se calculan ahora top y left?
- b) ¿Por qué se modificó la referencia si el contenedor no fue desplazado?

31. Compare relativo y absoluto. A partir de lo observado, responda:

- a) ¿Cuál de los dos tipos de posicionamiento conserva el espacio original del elemento?
- b) ¿Cuál permite que las demás cajas ocupen ese espacio?
- c) ¿Cómo se determina el sistema de referencia de un elemento absoluto?

32. Restituya el flujo normal. Cambie temporalmente la segunda tarjeta a:

position: static;

Observe que top y left dejan de producir efecto porque un elemento estático no se considera posicionado. Después de comprobarlo, elimine por completo la regla experimental de .maravilla:nth-child(2) y la declaración temporal aplicada a .maravillas.

Aplicación del posicionamiento absoluto

33. Coloque las etiquetas sobre las fotografías. Las etiquetas `.continente` deben quedar sobre cada imagen. Utilice lo aprendido:

- a) Convierta `.maravilla-imagen` en el antecesor posicionado sin desplazarlo.
- b) Extraiga `.continente` del flujo normal.
- c) Sepárelo 1rem desde arriba y 1rem desde la derecha.

Visualice el resultado antes de aplicar colores.

34. Complete la apariencia de las etiquetas. Agregue a `.continente`:

```
z-index: 1;
padding: 0.25rem 0.75rem;
color: var(--color-superficie);
background-color: var(--color-acentos);
border-radius: 0.3rem;
font-size: 0.8rem;
font-weight: bold;
```

Posicionamiento adherente y orden de superposición

35. Pruebe `position sticky` sin coordenada. Agregue al encabezado:

```
position: sticky;
```

Desplace la página.

- a) ¿Permanece adherido el encabezado?
- b) ¿Qué información falta para indicar el punto de adhesión?

36. Establezca el punto de adhesión. Agregue: `top: 0;`

Desplace nuevamente la página. Observe qué ocurre cuando las etiquetas de las tarjetas pasan cerca del encabezado.

37. Controle la superposición. Agregue `z-index: 10` al encabezado.

Compare el resultado con el paso anterior. Explique por qué el encabezado necesita un valor superior al utilizado en las etiquetas.

Posicionamiento fijo y modelo de caja

38. Fije el botón para volver al inicio. Localice .volver-arriba en el HTML y agregue:

```
.volver-arriba {  
  position: fixed;  
  right: 1rem;  
  bottom: 1rem;  
  z-index: 20;  
}
```

Desplace la página y compare el botón con el encabezado.

- a) ¿Cuál conserva su espacio original?
- b) ¿Cuál permanece ubicado respecto de la ventana desde el inicio?

39. Defina la forma del botón. Complete la regla del botón con:

```
display: flex;  
justify-content: center;  
align-items: center;  
width: 48px;  
height: 48px;  
color: var(--color-superficie);  
background-color: var(--color-acento);  
border: 2px solid var(--color-superficie);  
border-radius: 50%;  
font-size: 1.5rem;  
text-decoration: none;
```

Sección histórica y pie de página

40. Organice la sección histórica. Asigne a .seccion-historia un relleno vertical de 3rem y fondo --color-fondo-secundario.

Convierta .historia-contenido en un contenedor flexible que:

- a) Permita el salto de línea.
- b) Mantenga 2rem de separación.
- c) Alinee los elementos en el centro del eje transversal.

Haga que sus hijos directos puedan crecer, reducirse y partir de 300px.

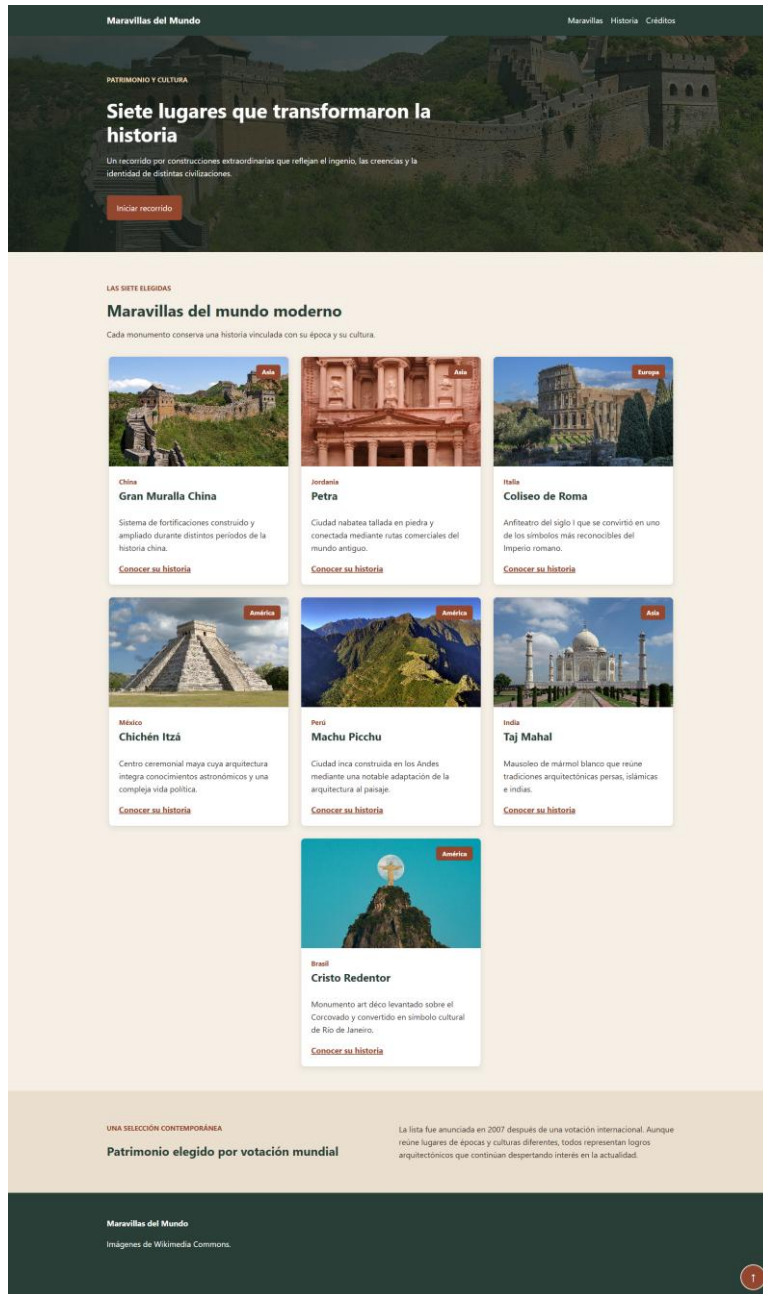
41. Formatee el pie. Configure .pie-pagina con:

- a) Relleno 2rem 0 5rem.
- b) Texto de color --color-superficie.
- c) Fondo --color-principal.

Verificación final

42. Revise la página. Compruebe:

- a) Las siete tarjetas se reorganizan al cambiar el ancho de la ventana.
- b) Las fotografías mantienen su proporción.
- c) Los enlaces de una misma fila aparecen alineados.
- d) Cada etiqueta se encuentra sobre su fotografía.
- e) El encabezado permanece adherido y por encima de las tarjetas.
- f) El botón conserva su posición respecto de la ventana.
- g) Los destinos internos no quedan ocultos bajo el encabezado.
- h) Las siete tarjetas están visibles.



EJEMPLO 2. MENU LATERAL DESPLEGABLE

En este ejemplo se construirá un menú lateral que se abre y se cierra sin JavaScript. El estado del menú se controlará con un checkbox y CSS responderá a ese estado mediante la pseudoclase :checked. Durante el procedimiento recordaremos el funcionamiento de los selectores estudiados en la unidad I.

Estructura HTML

1. Dentro de la carpeta ejemplo2, cree el archivo index.html con el siguiente contenido:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Compañía Miau</title>
  <link rel="stylesheet"
        href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-
awesome.min.css">
  <link rel="stylesheet" href="css/estilo.css">
</head>
<body>
  <input type="checkbox" name="checkMenu" id="checkMenu">

  <label for="checkMenu" class="labelBotonMenu">
    <i class="fa fa-bars" id="btnMostrar"></i>
    <i class="fa fa-times" id="btnOcultar"></i>
  </label>

  <aside class="sidebar">
    <header>
      <div class="logo">
        
      </div>
      <div class="titulo">Compañía Miau</div>
    </header>

    <nav aria-label="Navegación principal">
      <ul>
        <li><a href="#"><i class="fa fa-home"></i>Inicio</a></li>
        <li><a href="#"><i class="fa fa-shopping-bag"></i>Productos</a></li>
        <li><a href="#"><i class="fa fa-calendar"></i>Actividades</a></li>
        <li><a href="#"><i class="fa fa-link"></i>Enlaces</a></li>
        <li><a href="#"><i class="fa fa-cog"></i>Configuración</a></li>
        <li><a href="#"><i class="fa fa-question-circle-o"></i>Acerca de</a></li>
        <li><a href="#"><i class="fa fa-envelope-o"></i>Contacto</a></li>
      </ul>
    </nav>
  </aside>

  <main class="container"></main>
</body>
</html>
```

2. Identifique los elementos que controlarán el menú:

- a) El input de tipo checkbox almacenará dos estados: marcado y no marcado.
- b) El label está relacionado con el checkbox mediante `for="checkMenu"`. Al seleccionar cualquiera de sus iconos, cambia el estado del checkbox.
- c) El aside contiene el menú lateral. El main representa el contenido principal que se desplazará cuando se abra el menú.

El orden de estos elementos es importante. Los combinadores `+` y `~` solo pueden seleccionar hermanos que aparecen después del checkbox; por eso el input se coloca antes del label, del aside y del main.

Estilos iniciales

3. Dentro de `ejemplo2/css`, cree `estilo.css` y agregue los estilos iniciales:

```
@import
url("https://fonts.googleapis.com/css2?family=Montserrat:wght@400;600&display=swap");

:root {
  --ancho-menu: 250px;
  --color-menu: #042331;
  --color-encabezado: #063146;
  --color-texto: #ffffff;
  --color-secundario: #999999;
  --color-borde-claro: rgba(255, 255, 255, 0.1);
  --color-borde-oscuro: #000000;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: "Montserrat", Arial, sans-serif;
  font-size: 1rem;
}

a {
  text-decoration: none;
}
```

El selector universal `*` aplica el modelo de caja y elimina los márgenes y rellenos predeterminados de todos los elementos. Con `box-sizing: border-box`, el ancho declarado incluye el contenido, el relleno y el borde. Las variables permiten reutilizar los colores y el ancho del menú sin repetir valores en toda la hoja.

Construcción del menú lateral

4. Agregue la regla base del menú:

```
.sidebar {  
  position: fixed;  
  top: 0;  
  z-index: 20;  
  width: var(--ancho-menu);  
  background-color: var(--color-menu);  
  transition: left 0.3s;  
}
```

position: fixed extrae el menú del flujo normal y lo mantiene con respecto a la ventana. top: 0 fija su borde superior; z-index lo coloca por encima del fondo. La transición se aplica únicamente a left, que será la propiedad modificada para abrirlo.

5. Complete la regla de estilo anterior para que el menú mida toda la altura de la ventana y comience oculto exactamente 250px hacia la izquierda. Utilice height: 100vh y la propiedad left. Después cambie temporalmente left entre 0 y -250px para comprobar el desplazamiento; al finalizar, conserve -250px.

6. Formatee el encabezado, el logotipo y el nombre de la empresa:

```
.sidebar > header {  
  display: flex;  
  align-items: center;  
  min-height: 70px;  
  color: var(--color-texto);  
  background-color: var(--color-encabezado);  
  border-top: 1px solid var(--color-borde-claro);  
  border-bottom: 1px solid var(--color-borde-oscuro);  
  user-select: none;  
}  
  
.logo {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  width: 70px;  
}  
  
.logo img {  
  width: 52px;  
  height: 52px;
```

```

    object-fit: cover;
    border-radius: 50%;
}

.titulo {
    font-size: 1.05rem;
    font-weight: 600;
}

```

En `.sidebar > header`, el símbolo `>` exige que `header` sea hijo directo de `.sidebar`. `object-fit: cover` conserva la proporción del logotipo dentro del área de 52px por 52px y recorta lo que no cabe, evitando que la imagen se deforme.

7. Organice la lista y dé formato a los enlaces del menú:

```

.sidebar ul {
    list-style: none;
}

.sidebar ul a {
    display: block;
    padding: 1.1rem 1.5rem 1.1rem 2.5rem;
    color: var(--color-texto);
    border-top: 1px solid var(--color-borde-claro);
    border-bottom: 1px solid var(--color-borde-oscuro);
    transition: padding-left 0.3s;
}

.sidebar ul a i {
    margin-right: 10px;
}

```

8. Cree el efecto de desplazamiento al pasar el puntero. Escriba una regla que seleccione únicamente el enlace hijo directo de un `li` en estado `:hover` dentro de `.sidebar` y cambie su `padding-left` a 3.1rem.

El selector esperado comienza con `.sidebar li:hover` y utiliza `> a`. `:hover` representa el estado temporal del `li` cuando el puntero se encuentra sobre él; `>` evita que la regla alcance enlaces que pudieran existir dentro de niveles de menú más profundos.

Control de apertura y cierre

9. Oculte el checkbox y configure la posición inicial de los dos iconos:

```

#checkMenu {
    display: none;
}

```

```

#btnMostrar,
#btnOcultar {
  position: fixed;
  z-index: 30;
  cursor: pointer;
  background-color: var(--color-menu);
  border-radius: 3px;
  transition: left 0.3s, opacity 0.3s;
}

#btnMostrar {
  top: 25px;
  left: 40px;
  padding: 6px 12px;
  color: var(--color-texto);
  font-size: 35px;
}

#btnOcultar {
  top: 17px;
  left: -200px;
  padding: 4px 9px;
  color: var(--color-secundario);
  font-size: 30px;
}

```

display: none impide que el checkbox se dibuje, pero el label todavía puede cambiar su estado por medio del atributo for. Inicialmente, el icono para abrir está visible y el icono para cerrar permanece fuera de la ventana.

10. Haga visible el menú cuando el checkbox esté marcado:

```

#checkMenu:checked ~ .sidebar {
  left: 0;
}

```

El selector se interpreta de izquierda a derecha: #checkMenu localiza el checkbox; :checked exige que esté marcado; ~ busca un hermano posterior con la clase .sidebar. Cuando la condición se cumple, left cambia de -250px a 0 y la transición produce el movimiento. Si .sidebar apareciera antes del checkbox o tuviera otro padre, este selector no funcionaría.

11. Cambie los iconos según el estado del checkbox:

```
#checkMenu:checked + .labelBotonMenu #btnMostrar {
  left: var(--ancho-menu);
  opacity: 0;
  pointer-events: none;
}

#checkMenu:checked + .labelBotonMenu #btnOcultar {
  left: 200px;
}
```

El combinador + selecciona .labelBotonMenu porque es el hermano inmediatamente posterior del checkbox. El espacio antes de #btnMostrar o #btnOcultar es un selector descendiente: busca cada icono dentro del label seleccionado. Al abrir el menú, el icono de barras se vuelve transparente y deja de recibir clics; el icono de cierre se desplaza hasta left: 200px.

Resumen de los selectores utilizados

12. Compare la función de cada forma de selector antes de continuar:

Forma	Tipo	Interpretación en el ejemplo
#checkMenu	Selector de identificador	Selecciona el checkbox cuyo id es checkMenu.
:checked	Pseudoclase de estado	Solo coincide cuando el checkbox está marcado.
A ~ B	Hermano general	Selecciona B si aparece después de A y ambos comparten el mismo padre.
A + B	Hermano adyacente	Selecciona B únicamente si aparece inmediatamente después de A.
A B	Descendiente	Selecciona B cuando se encuentra dentro de A, aunque existan niveles intermedios.
A > B	Hijo directo	Selecciona B solamente cuando su padre inmediato es A.
A, B	Agrupación	Aplica el mismo bloque de declaraciones a A y a B.

Contenido principal

13. Agregue la imagen de fondo al contenido principal:

```
.container {  
  min-height: 100vh;  
  background-image: url("../img/fondo-bosque.jpg");  
  background-repeat: no-repeat;  
  background-position: center;  
  background-size: cover;  
  transition: margin-left 0.3s;  
}
```

min-height: 100vh garantiza que el contenido tenga, como mínimo, la altura completa de la ventana. background-size: cover hace que la fotografía cubra toda el área conservando su proporción, aunque una parte pueda quedar recortada.

14. Desplace el contenido cuando el menú esté abierto. Aplique lo aprendido: seleccione .container cuando sea un hermano posterior del checkbox marcado y asigne margin-left: var(--ancho-menu).

El selector debe utilizar la misma condición #checkMenu:checked y el combinador ~. Así, el menú y el contenido responden al mismo estado sin necesitar JavaScript. La transición de margin-left sincroniza visualmente el desplazamiento del fondo con la apertura del menú.

Cómo funciona el menú

15. Siga la secuencia completa con el checkbox desmarcado:

- a) .sidebar conserva left: -250px y queda fuera del área visible.
- b) #btnMostrar permanece en left: 40px y permite abrir el menú.
- c) #btnOcultar permanece en left: -200px y .container no tiene margen izquierdo.

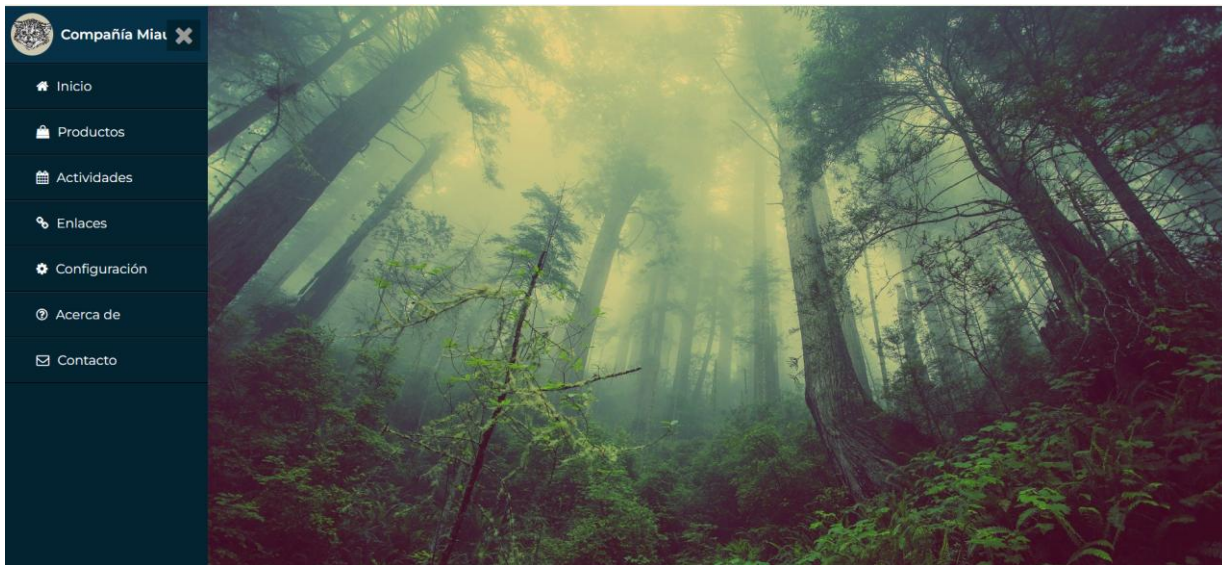
16. Seleccione el icono de barras y siga nuevamente la secuencia:

- a) El label cambia el estado de #checkMenu porque su atributo for coincide con el id del checkbox.
- b) :checked comienza a coincidir y activa las tres reglas dependientes del estado.
- c) ~ desplace .sidebar hasta left: 0 y mueve .container mediante margin-left.
- d) + localiza el label inmediato; dentro de él, los selectores descendientes intercambian los iconos.

Verificación final

17. Revise la página y compruebe:

- a) La marca visible en el encabezado dice Compañía Miau.
- b) Al cargar la página, el menú está oculto y aparece el icono de barras.
- c) Al seleccionar el icono, el menú entra desde la izquierda y el contenido se desplaza 250px.
- d) El icono de barras desaparece y el icono de cierre queda visible sobre el menú.
- e) Al seleccionar el icono de cierre, todos los elementos regresan a su posición inicial.
- f) Los enlaces se desplazan ligeramente a la derecha durante :hover.
- g) El menú conserva toda la altura de la ventana y el logotipo no se deforma.



IV. Ejercicios complementarios (20%)

1. Dentro de los recursos de la guía se le proporciona un archivo llamada “ejercicio.html”. Utilizar una hoja de estilo externa y aplicar las reglas de estilo necesarias para conseguir los siguientes resultados:

Resultado 1

Usando posicionamiento y z-index



Resultado 2

Usando posicionamiento y z-index



Nota: Debe usar propiedades de posicionamiento y la propiedad z-index

2. Publique los ejemplos y ejercicios de esta guía en su repositorio de guías de la asignatura. Actualice el index.html de su repositorio para incluir los enlaces hacia los ejemplos y ejercicios.

Fecha de entrega: sábado 26 de septiembre

V. Reto IA (30%)

En este reto utilizará una herramienta de inteligencia artificial para crear una nueva versión de la página de inicio del sitio web salvadoreño desarrollado en las guías anteriores. La página deberá conservar la temática, el contenido esencial y la navegación del sitio, pero su distribución principal deberá construirse mediante CSS Grid.

La herramienta seleccionada debe ser diferente de las utilizadas en las guías prácticas 1 y 2. Deberá revisar el código generado y comprender las propiedades utilizadas, pues posteriormente explicará su funcionamiento mediante un video.

Procedimiento

1. **Selección de la herramienta.** Seleccione una herramienta de inteligencia artificial distinta de las utilizadas en las guías prácticas 1 y 2.
2. **Preparación de la instrucción.** Redacte una instrucción detallada para solicitar una nueva versión de la página de inicio. La instrucción debe especificar:
 - a) La temática y finalidad del sitio.
 - b) El contenido que debe conservarse.
 - c) La identidad visual que debe mantenerse respecto de las demás páginas.
 - d) El uso obligatorio de CSS Grid para construir la distribución principal.
 - e) La generación separada del código HTML y CSS.
3. **Uso obligatorio de CSS Grid.** La distribución principal de la página debe utilizar CSS Grid. No será suficiente emplearlo únicamente en una galería, un conjunto de tarjetas u otro componente aislado. Puede utilizar Flexbox dentro de componentes específicos, como el menú o los botones.
4. **Revisión del resultado.** Ejecute el código generado y compruebe su funcionamiento. Revise las rutas, los enlaces, el contenido, la estructura HTML y las reglas CSS. Realice las correcciones necesarias antes de publicar la página.
5. **Conservación de evidencias.** Guarde:
 - a) La instrucción enviada a la herramienta.
 - b) El código originalmente generado.
 - c) El código final después de las correcciones.
 - d) Una breve lista de los cambios efectuados sobre el resultado original.
6. **Grabación del video.** Grabe un video de hasta diez minutos y publíquelo en YouTube o una plataforma similar. En el video debe:
 - a) Identificar la herramienta de inteligencia artificial utilizada.
 - b) Mostrar brevemente la instrucción proporcionada.

- c) Presentar la página publicada.
- d) Identificar el contenedor Grid y los elementos que actúan como Grid items.
- e) Explicar cómo funcionan las propiedades de CSS Grid presentes en su código, incluyendo la definición de columnas o filas, la separación entre elementos y su ubicación.
- f) Modificar en vivo al menos una propiedad relacionada con Grid, explicar previamente qué cambio espera observar y comprobar el resultado en el navegador.